

Politechnika Świętokrzyska w Kielcach
Wydział Elektrotechniki, Automatyki i Informatyki
Katedra Informatyki Stosowanej

Technologie Obiektowe
Projekt (studia stacjonarne 2 stopnia)
Informatyka, rok akademicki - 2025/2026

Temat projektu:

Web-owe IDE do dynamicznej kompilacji i uruchamiania kodu wraz
z modułem zdalnego wdrażania

Wykonali:
Bartosz Kaczmarczyk Grupa:
1ID21A

1. Wprowadzenie

Aplikacja webowa pozwala na pisanie, kompilowanie i uruchamianie kodu źródłowego (Java, Python, C++) bezpośrednio z poziomu przeglądarki, z wynikami przesyłanymi w czasie rzeczywistym przez WebSocket. System pozwala także na wdrażanie skompilowanych programów na zdalne serwery za pomocą protokołu SSH/SFTP oraz na zdalne wykonywanie poleceń. Projekt podzielony jest na dwa główne moduły: backend (Java 21 / Spring Boot) odpowiedzialny za zapis kodu, kompilację, uruchamianie procesów, transfer plików SSH oraz powiadomienia WebSocket, oraz frontend (React 19 + TypeScript + Vite) zorganizowany według wzorca MVVM, z modułami: editor, deploy, execution, process (upload) oraz terminal.

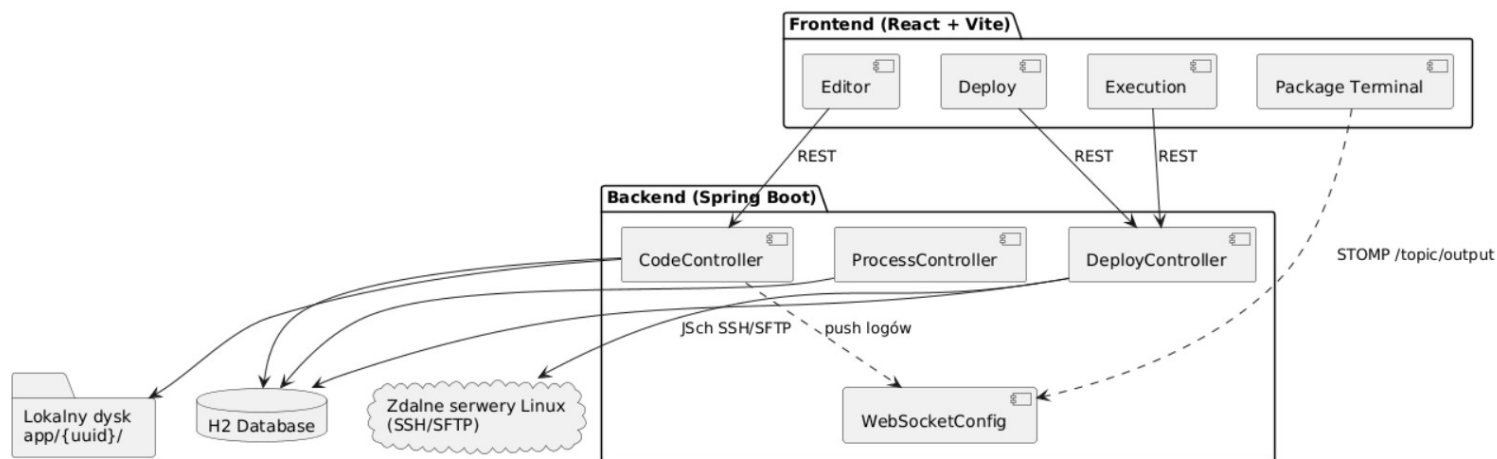
Niniejszy dokument zawiera opis architektury systemu, zastosowanych wzorców projektowych, struktury modułów backendu i frontendu, kontraktu API REST, modelu bazy danych oraz przepływu danych pomiędzy komponentami (w tym komunikacji WebSocket)

2. Stos technologiczny

Warstwa	Technologia	Zastosowanie
Backend	Java 21, Spring Boot	Logika biznesowa, REST API
Backend	Spring WebSocket (STOMP/SockJS)	Strumieniowanie logów procesów na żywo
Backend	Spring Data JPA + H2	Persystencja danych (kod, konfiguracje, serwery)
Backend	JSch	Połączenia SSH/SFTP do zdalnych serwerów
Backend	Lombok, Springdoc OpenAPI	Redukcja boilerplate, dokumentacja Swagger
Backend	Maven	Budowanie projektu (mvnw)
Frontend	React 19 + TypeScript	SPA, komponenty UI
Frontend	Vite	Bundler / dev server
Frontend	react-simple-code-editor + PrismJS	Edytor kodu z podświetlaniem składni
Frontend	SockJS + StompJS	Połączenie WebSocket z backendem

3. Architektura systemu

Poniższy diagram przedstawia ogólną architekturę systemu — przepływ żądań od frontendu przez REST API i WebSocket, do backendu, bazy danych H2, lokalnego systemu plików oraz zdalnych serwerów docelowych.

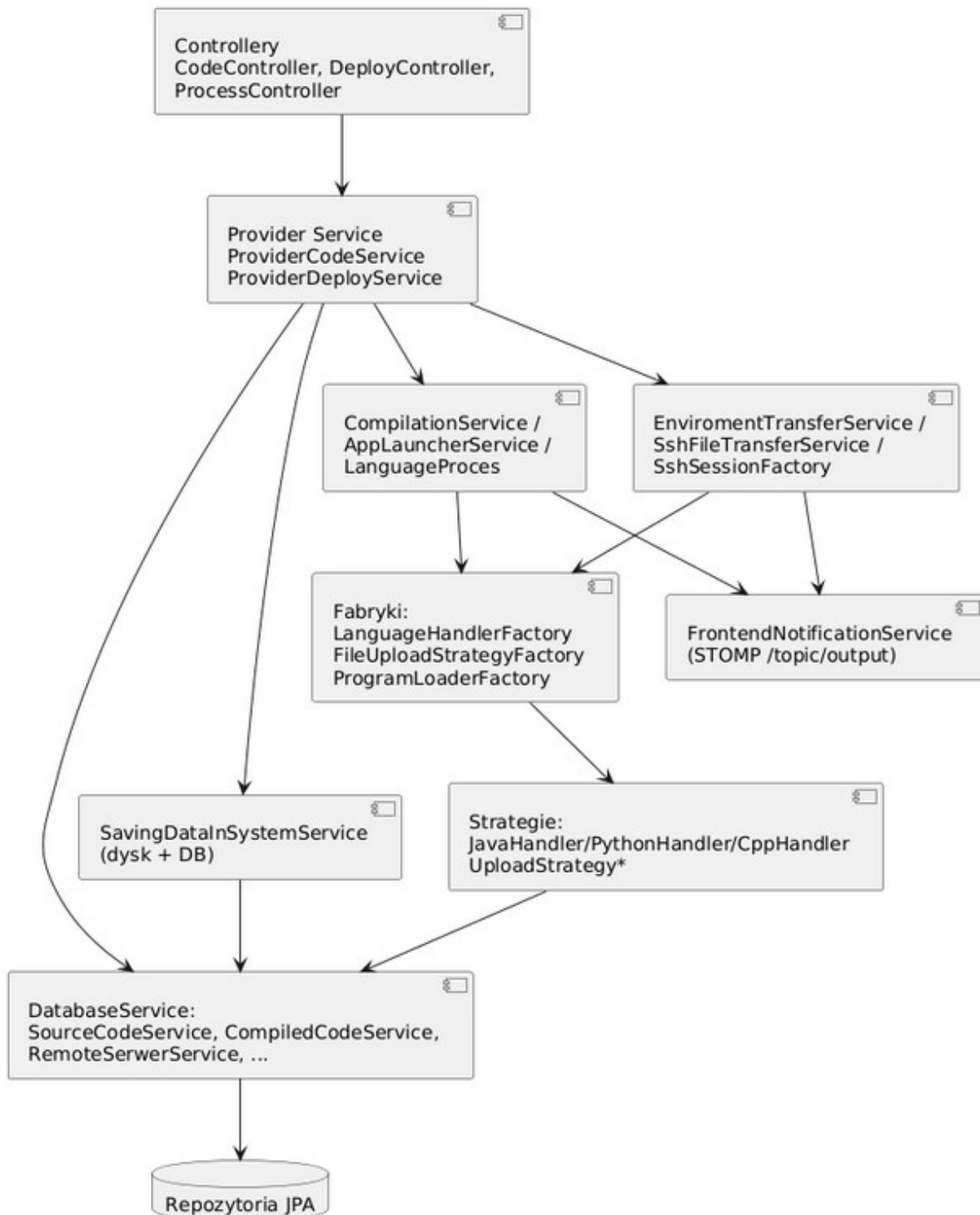


3.1. Opis przepływu

- Moduł Editor (frontend) komunikuje się z CodeController – zapis kodu źródłowego, kompilacja, uruchamianie.
- Moduł Deploy komunikuje się z DeployController – zarządzanie serwerami SSH oraz transfer plików (źródła / pliki skompilowane / pliki wykonywalne).
- Moduł Execution korzysta z DeployController – odczyt zarejestrowanych programów (RemoteProgramConfiguration) oraz wykonywanie poleceń na zdalnym serwerze.
- PackageTerminal subskrybuje kanał STOMP /topic/output, na który backend publikuje logi z procesów kompilacji, uruchamiania oraz operacji SSH.
- Dane zapisywane są w bazie H2 (encje SourceCode, CompiledCode, RemoteServerConfiguration, RemoteProgramConfiguration, RemoteExecutable, foldery na dysku) oraz na lokalnym dysku w katalogu app/{uuid}/.

4. Struktura backendu (App-Instance-Manager)

Backend zorganizowany jest w warstwy: kontrolery REST, serwisy „Provider” agregujące logikę, serwisy domenowe (kompilacja, transfer SSH, zapis danych), fabryki i strategie (wzorce projektowe) oraz warstwę dostępu do danych (DatabaseService + repozytoria JPA).



4.1. Pakiety i odpowiedzialności

Pakiet	Odpowiedzialność
controller.code	CodeController – REST API zapisu/odczytu kodu źródłowego, kompilacji, uruchamiania
controller.deploy	DeployController – serwery, programy, transfer SSH, wykonywanie komend
controller.process	ProcessController – CRUD plików wykonywalnych (RemoteExecutable)
controller.test.*	Endpointy pomocnicze profilu „dev” (kompilator, deploy, konfiguracja, SSH)
service.code.compilation	CompilationService – kompilacja wielu plików dla głównej klasy projektu
service.code.launcher	AppLauncherService – uruchamianie skompilowanej/zinterpretowanej aplikacji
service.code.prcoces	LanguageProces – uruchamianie procesów (ProcessBuilder), strumieniowanie logów
service.code.provider	ProviderCodeService – fasada łącząca zapis, kompilację i uruchamianie
service.deploy.connection	SshSessionFactory – tworzenie sesji SSH (JSch), wykonywanie komend
service.deploy.transport	EnviromentTransferService, SshFileTransferService – transfer plików SFTP wg strategii
service.deploy.serwer	SerwerConfigurationService – rejestracja serwera + automatyczna detekcja OS
service.deploy.execution	ExecutionService – wykonanie polecenia na zdalnym serwerze (cd + komenda)
service.file	SavingDataInSystemService – zapis kodu na dysk + DB, obsługa wyników kompilacji
fileutil	FileService – operacje na plikach (zapis/odczyt, generowanie unikalnej ścieżki UUID)
mapper	MapperCode, MapperFile – mapowanie encji SourceCode ↔ CompiledCode ↔ foldery na dysku
factory.code.langugage	LanguageHandlerFactory – wybór handlera języka wg rozszerzenia pliku
factory.deploy.sshfiiletransfer	FileUploadStrategyFactory – wybór strategii transferu wg UploadStrategyType
factory.deploy.provider	ProgramLoaderFactory – wybór dostawcy danych wg strategii uploadu
strategy.language	JavaHandler, PythonHandler, CppHandler – komendy kompilacji/uruchomienia per język
strategy.sshfiiletransfer	SourceCodeUploadStrategy, CompiledCodeUploadStrategy, RemoteExecutableUploadStrategy
strategy.memory	MemorySourceFile, MemoryByteCodeFile, MemoryJavaFileManager – kompilacja Javy w pamięci
databaseservice repository /	Warstwa DAO – serwisy CRUD nad encjami JPA (SourceCode, CompiledCode, serwery, programy, executables, foldery)
notification	FrontendNotificationService – wysyłanie komunikatów na kanał STOMP /topic/output
constants / model	Enumy (LanguageType, UploadStrategyType, OperationSystem) oraz encje / DTO

5. Zastosowane wzorce projektowe

Wzorzec	Klasy	Opis zastosowania
Strategy	LanguageHandler (JavaHandler, PythonHandler, CppHandler)	Każdy język ma własną implementację generowania komend kompilacji i uruchomienia, co umożliwia łatwe dodawanie kolejnych języków.
Strategy	FileUploadStrategy (SourceCodeUploadStrategy, CompiledCodeUploadStrategy, RemoteExecutableUploadStrategy)	Sposób pozyskania zawartości plików do wysłania przez SFTP zależy od wybranej strategii (kod źródłowy / skompilowany / plik wykonywalny).
Factory	LanguageHandlerFactory, FileUploadStrategyFactory, ProgramLoaderFactory	Mapy Spring (Map<String, Bean>) wstrzykiwane przez konstruktor; wybór odpowiedniej strategii/handlera w czasie wykonania.
Facade / Provider	ProviderCodeService, ProviderDeployService	Ujednolicony interfejs dla kontrolerów, agregujący wiele serwisów niższego poziomu.
Repository / DAO	Spring Data JPA (CodeRepository, CompiledCodeRepository, RemoteServerRepository, ...)	Standardowy dostęp do bazy H2 z dodatkowymi zapytaniami JPQL (np. pobranie głównych klas projektów).
MVVM (frontend)	model / viewmodel / views w każdym module (editor, deploy, execution, process)	Separacja typów danych, logiki (hooki React) i prezentacji (komponenty).
Observer / Pub-Sub	FrontendNotificationService + STOMP/SockJS, PackageTerminal	Backend publikuje zdarzenia (logi procesów, statusy SSH) na kanał /topic/output, frontend subskrybuje i wyświetla na bieżąco.

6. Kontrakt API REST

6.1. Moduł kodu (CodeController, /api/code)

Metoda	Endpoint	Opis
GET	/api/code/source/main-class/{idMainClass}	Pobranie plików źródłowych projektu
POST	/api/code/source	Zapis listy plików źródłowych (kod + metadane) na dysk i do DB
PUT	/api/code/source	Aktualizacja pojedynczego pliku źródłowego
GET	/api/code/compiled/main-class/{idMainClass}	Pobranie skompilowanych plików (.class itp.) projektu
DELETE	/api/code/project/{idMainClass}	Usunięcie całego projektu (źródła + skompilowane)
GET	/api/code/source/all	Pobranie wszystkich plików źródłowych z bazy
GET	/api/code/source/main-class/all	Pobranie listy głównych klas (projektów)
POST	/api/code/compile/all/main-class/{idMainClass}	Kompilacja wszystkich plików projektu
GET	/api/code/source/folders/all	Lista folderów źródłowych zapisanych na dysku
POST	/api/code/launch/main-class/{idMainClass}	Uruchomienie skompilowanej/zinterpretowanej aplikacji

6.2. Moduł wdrożeń (DeployController, /api/deploy)

Metoda	Endpoint	Opis
GET	/api/deploy/servers	Lista skonfigurowanych serwerów SSH
POST	/api/deploy/server	Rejestracja nowego serwera (auto-detekcja systemu operacyjnego)
POST	/api/deploy/server/bulk	Masowa rejestracja serwerów
GET	/api/deploy/server/{idConfiguration}	Szczegóły konfiguracji serwera
DELETE	/api/deploy/server/{idConfiguration}	Usunięcie konfiguracji serwera
GET	/api/deploy/program/{mainClassId}	Konfiguracja programu wg ID głównej klasy
GET	/api/deploy/programs	Lista wszystkich skonfigurowanych programów (do wdrożenia/uruchomienia)
POST	/api/deploy/program/execute	Wykonanie polecenia na zdalnym serwerze (ExecuteDto: command, programConfigId)
POST	/api/deploy/transfer/bulk	Masowy transfer plików (lista TransferTask)
POST	/api/deploy/transfer/single	Pojedynczy transfer (mainClassId, idConfiguration, strategia)

6.3. Moduł procesów (ProcessController, /api/process)

Metoda	Endpoint	Opis
POST	/api/process/upload	Upload pliku wykonywalnego (multipart) wraz z językiem
POST	/api/process/bulk	Masowy zapis plików wykonywalnych
GET	/api/process	Lista wszystkich plików wykonywalnych
GET	/api/process/{id}	Pobranie pliku wykonywalnego po ID
PUT	/api/process/{id}	Aktualizacja pliku wykonywalnego

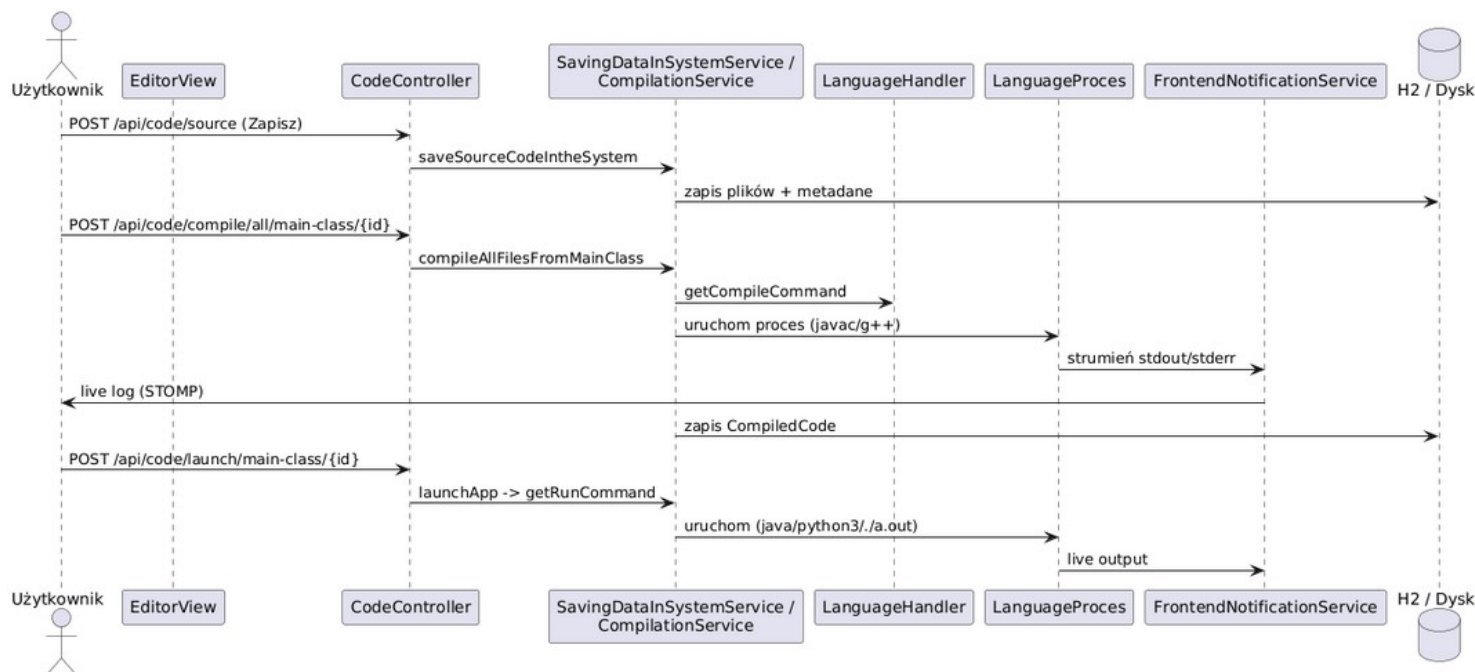
Metoda	Endpoint	Opis
DELETE	/api/process/{id}	Usunięcie pliku wykonywalnego
DELETE	/api/process	Usunięcie wszystkich plików wykonywalnych

6.4. Komunikacja WebSocket (STOMP/SockJS)

Element	Opis
Endpoint	/ws-console (SockJS, allowedOriginPatterns="*")
Broker	Prosty broker /topic (config.enableSimpleBroker)
Kanał	/topic/output – logi kompilacji, uruchamiania oraz operacji SSH (FrontendNotificationService.sendToFrontend)
Klient	PackageTerminal.tsx – panel „OUTPUT” z możliwością połączenia/rozłączenia, zwijania i zmiany wysokości

7. Przepływ: zapis → kompilacja → uruchomienie

Poniższy diagram sekwencji przedstawia typowy scenariusz pracy w module Editor: zapis plików projektu, kompilacja i uruchomienie, wraz z przesyłaniem logów na żywo przez WebSocket.



Kluczowe elementy implementacji:

- Każdy projekt identyfikowany jest przez idManClass – ID pierwszego (głównego) pliku, dla którego id == idManClass.
- Pliki zapisywane są na dysku w katalogu app/{uuid}/ (UUID generowany przez FileService.generateUniquePath).
- Kompilacja Javy odbywa się w pamięci (javax.tools.JavaCompiler + MemoryJavaFileManager), natomiast C++ kompilowany jest zewnętrznym kompilatorem g++, a Python nie wymaga kompilacji.
- Wynik procesu (stdout/stderr) jest odczytywany linia po linii i przesyłany na kanał /topic/output w czasie rzeczywistym.

8. Struktura frontendu (App-Instance-Manager-fronted)

Frontend zbudowany jest jako SPA w React + TypeScript, z własnym, lekkim routerem (useRouter) – bez biblioteki React Router. Każdy moduł funkcjonalny zorganizowany jest zgodnie ze wzorcem MVVM: model (typy), viewmodel (logika, hooki), views (komponenty prezentacyjne).

Moduł	Widok główny	Zakres odpowiedzialności
editor	ProjectsView, EditorView	Lista projektów, edycja kodu (Prism + react-simple-code-editor), zapis, kompilacja, uruchamianie
deploy	DeployView	Trójpanelowy widok: pliki źródłowe / panel centralny (plan i log wdrożenia) / serwery SSH
execution	ExecutionView	Lista wdrożonych instancji, wysyłanie poleceń (presety: Java, Python, Node, C/C++, System)
process	UploadView	Upload dowolnego pliku wykonywalnego (drag&drop) z wyborem języka/typu
terminal	PackageTerminal	Stały panel na dole ekranu – podgląd logów STOMP z możliwością zwiżania i zmiany rozmiaru
router	useRouter	Nawigacja pomiędzy ekranami: projects, editor, deploy, execution, upload

8.1. Kluczowe typy danych (frontend)

Typ	Opis
LanguageType	'JAVA' 'PYTHON' 'CPP' 'BASH' 'UNKNOWN' – odpowiada backendowemu enumowi LanguageType
UploadStrategyType	'SOURCE_CODE' 'COMPILED_CODE' 'REMOTE_EXECUTABLE' – strategia transferu plików
Project / ProjectFile	Reprezentacja projektu i pojedynczego pliku w edytorze (id, name, ext, content, language)
DeployServer / DeployFile	Konfiguracja serwera SSH oraz plik gotowy do wdrożenia wraz z przypisaną strategią
DeployedInstance	Zmapowana para RemoteProgramConfiguration + RemoteSerwerConfiguration – aktywna instancja na zdalnym serwerze

9. Konfiguracja i uruchomienie środowiska

9.1. Backend

- Build: Maven Wrapper (mvnw / mvnw.cmd), wersja Apache Maven 3.9.12.
- Plugin frontend-maven-plugin automatycznie instaluje Node 20.18.0, buduje frontend (npm install, npm run build) i kopiuje wynik do target/classes/static.
- Profil „dev” (@Profile("dev")) udostępnia dodatkowe kontrolery testowe (kompilator, deploy, konfiguracja SSH, transfer środowisk).
- Baza danych: H2 (dependency com.h2database:h2).

9.2. Frontend

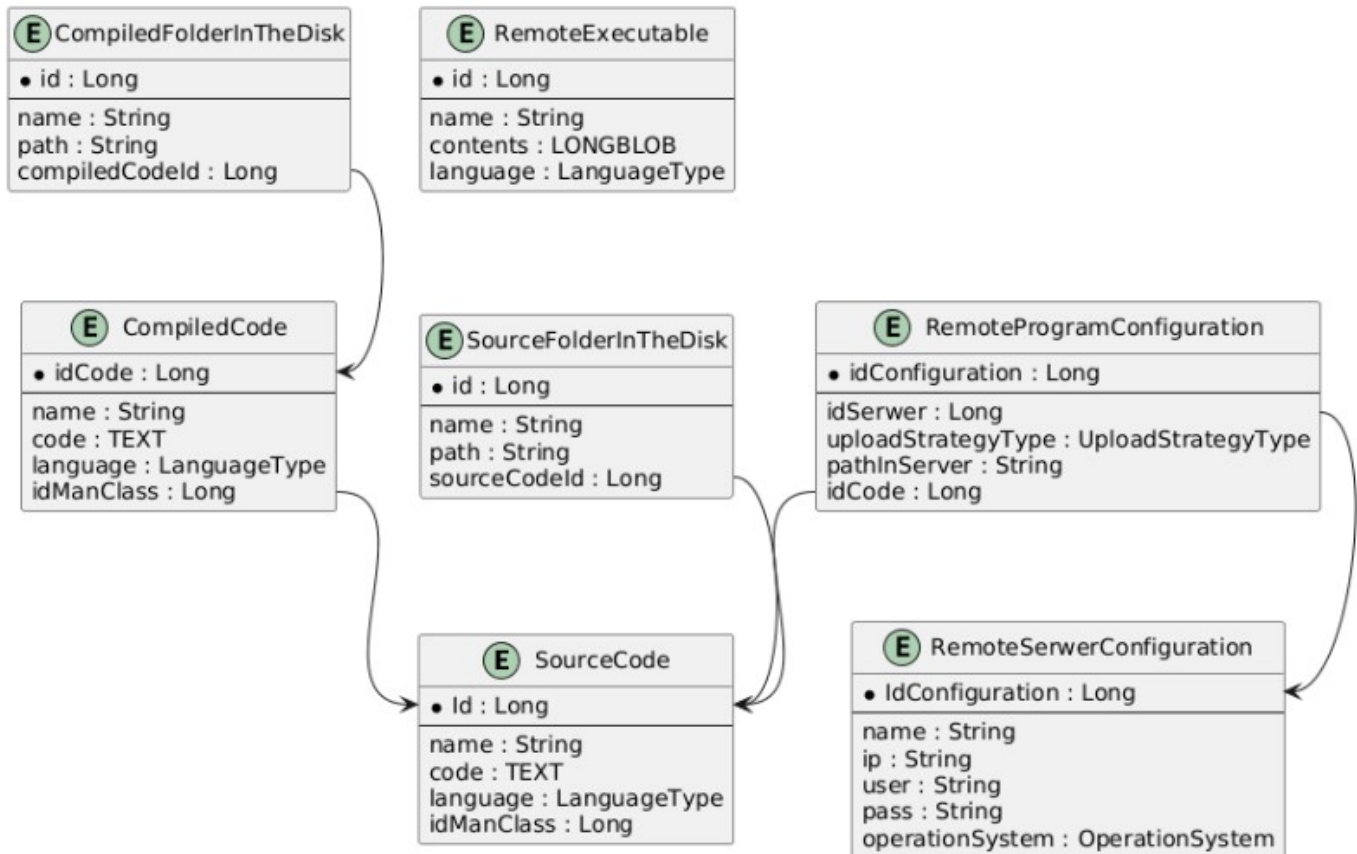
- Vite + React 19 + TypeScript, alias @ wskazujący na ./src.
- Edytor kodu: react-simple-code-editor + PrismJS (obsługa: TypeScript, JavaScript, Python, Rust, CSS, JSON).
- Adresy API skonfigurowane na http://localhost:8888 (REST) oraz http://localhost:8888/ws-console (WebSocket).

9.3. Środowisko testowe (Docker)

Plik docker-compose.yml definiuje 5 kontenerów Linux (linux-1 ... linux-5) opartych o obraz Alpine z zainstalowanym OpenSSH, OpenJDK 17 oraz Python 3 – służą jako cele testowe dla modułu wdrożeń SSH (porty 2221–2225, użytkownik „user”/hasło „user”).

10. Diagram bazy danych (ERD)

Baza danych H2 odzwierciedla encje JPA opisane w pakiecie model.entity. Kluczem grupującym pliki jednego projektu jest idManClass (ID pliku głównego, dla którego $Id == idManClass$). Poniżej kod diagramu encji i relacji (PlantUML, editor.plantuml.com).

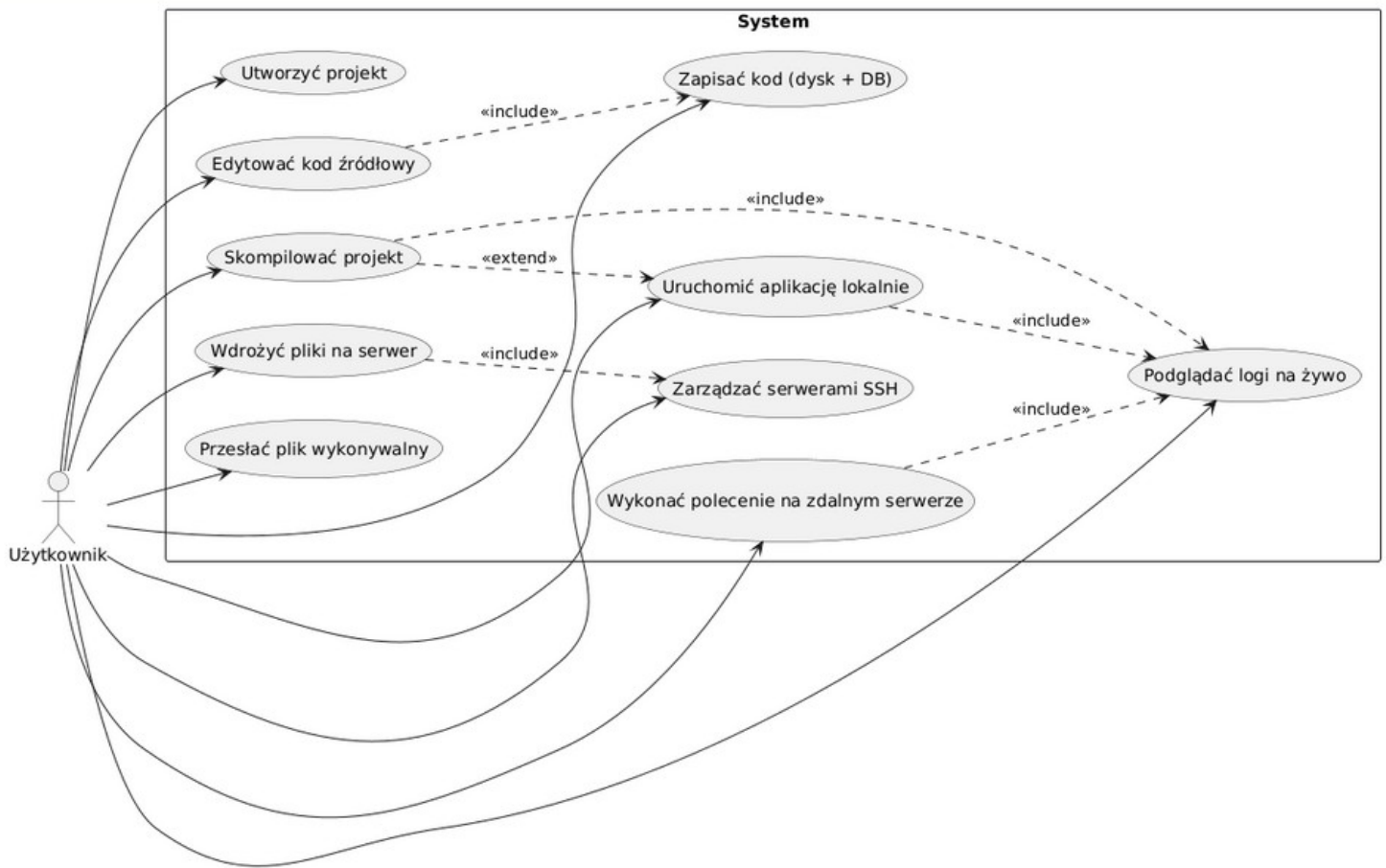


Opis relacji:

- SourceCode – pliki źródłowe projektu; wszystkie pliki danego projektu mają to samo idManClass, równe Id pliku głównego.
- CompiledCode – wynik kompilacji pliku źródłowego, powiązany z tym samym idManClass.
- SourceFolderInTheDisk / CompiledFolderInTheDisk – metadane folderu na dysku (UUID), powiązane 1:1 z projektem przez sourceCodeId / compiledCodeId.
- RemoteServerConfiguration – dane logowania SSH do zdalnego serwera (1:N do RemoteProgramConfiguration).
- RemoteProgramConfiguration – łączy projekt (idCode), serwer (idServer) oraz strategię wdrożenia (UploadStrategyType) wraz ze ścieżką docelową.
- RemoteExecutable – samodzielnie przesłane pliki binarne/wykonywalne (niezależne od projektów kodu źródłowego).

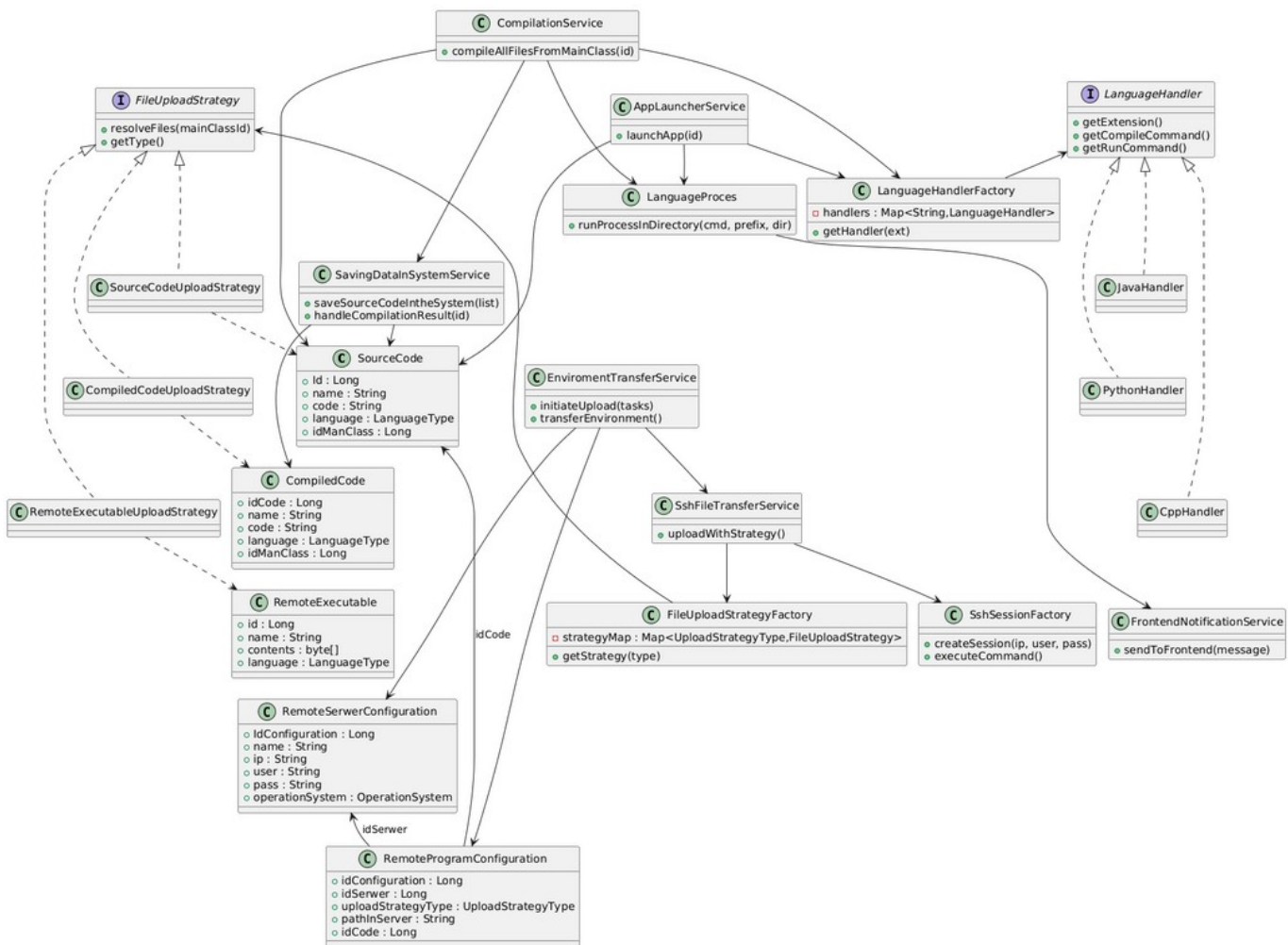
11. Diagram przypadków użycia (Use Case)

Diagram przedstawia główne przypadki użycia systemu z perspektywy jednego aktora — Użytkownika, wraz z relacjami <<include>> i <<extend>>.



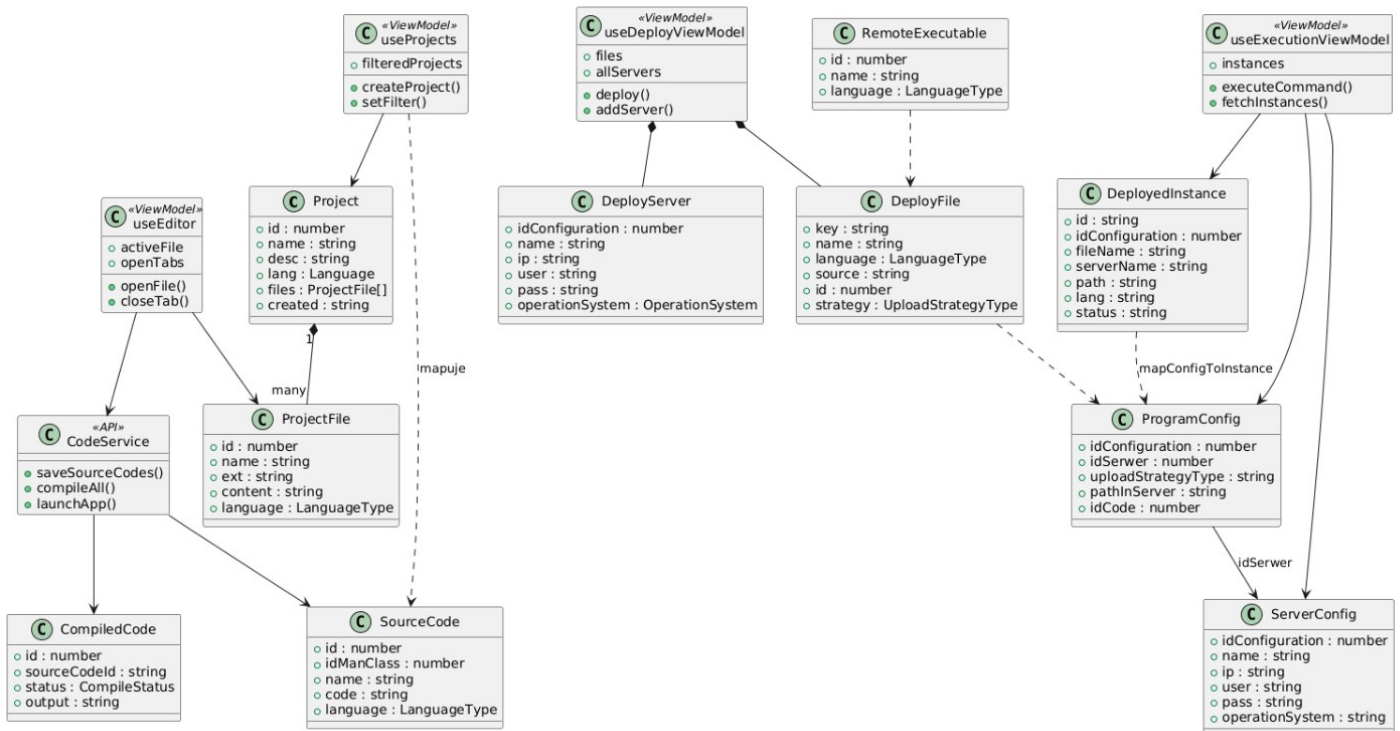
12. Diagram klas — backend (Java / Spring)

Diagram przedstawia kluczowe encje JPA, interfejsy strategii (LanguageHandler, FileUploadStrategy), fabryki oraz serwisy domenowe wraz z zależnościami między nimi.



13. Diagram klas — frontend (TypeScript)

Diagram przedstawia kluczowe typy/interfejsy TypeScript oraz hooki ViewModel wraz z relacjami między modelami danych a logiką poszczególnych modułów.



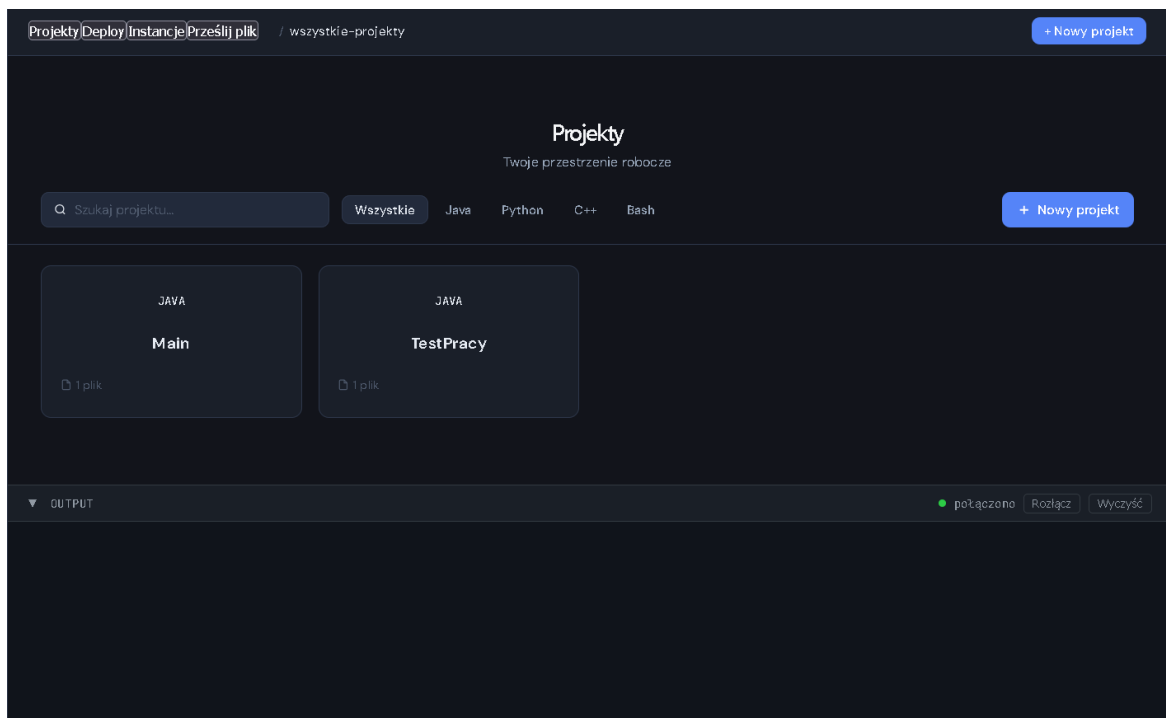
14. Diagram przepływu (Activity)

Diagram aktywności przedstawia ogólny przepływ pracy użytkownika: od utworzenia projektu, przez edycję, zapis, kompilację (jeśli wymagana), uruchomienie lokalne, aż po opcjonalne wdrożenie na zdalny serwer i wykonanie tam poleceń.

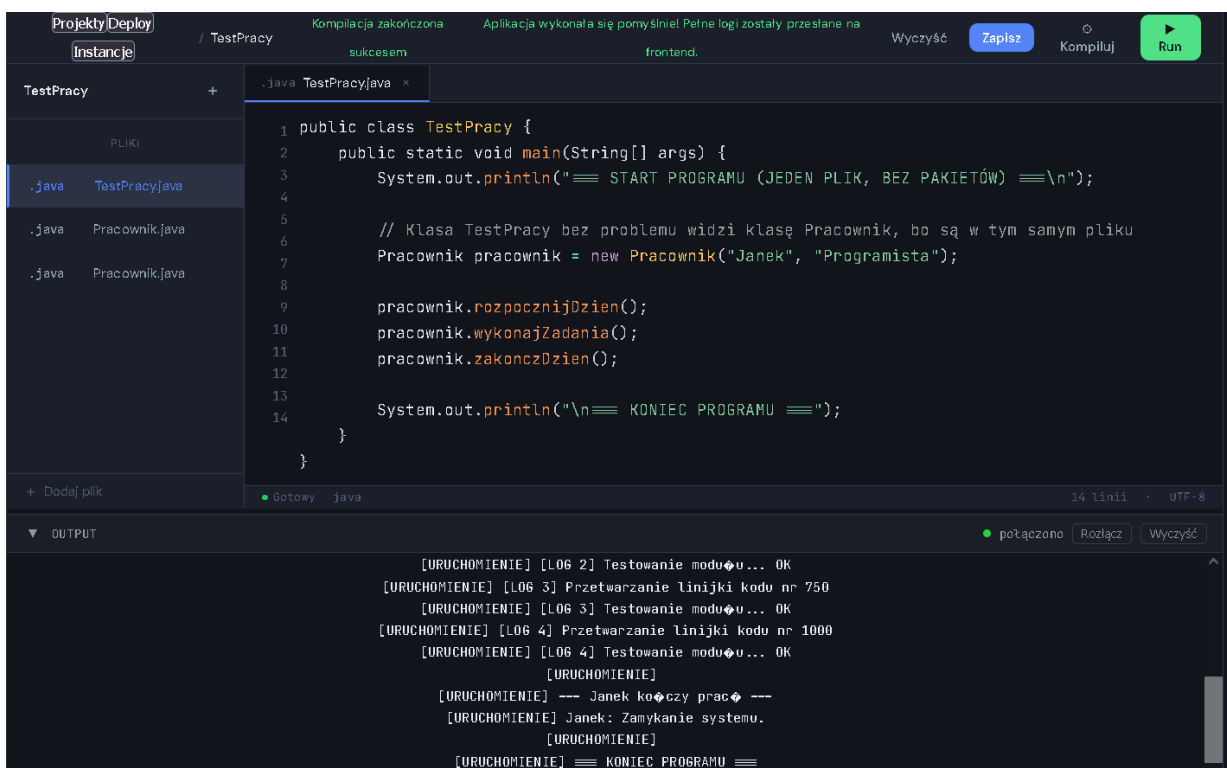


15 Wygląd aplikacji

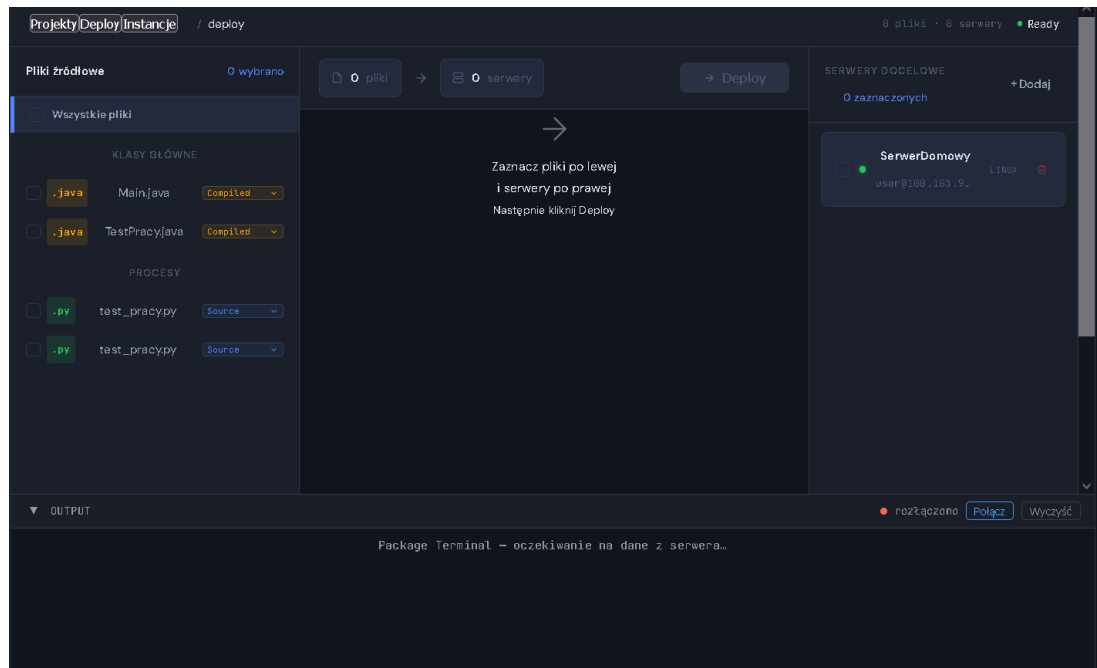
Projekty – widok startowy aplikacji. Lista projektów użytkownika z podziałem na język (Java, Python, C++, Bash), wyszukiwarką oraz opcją utworzenia nowego projektu



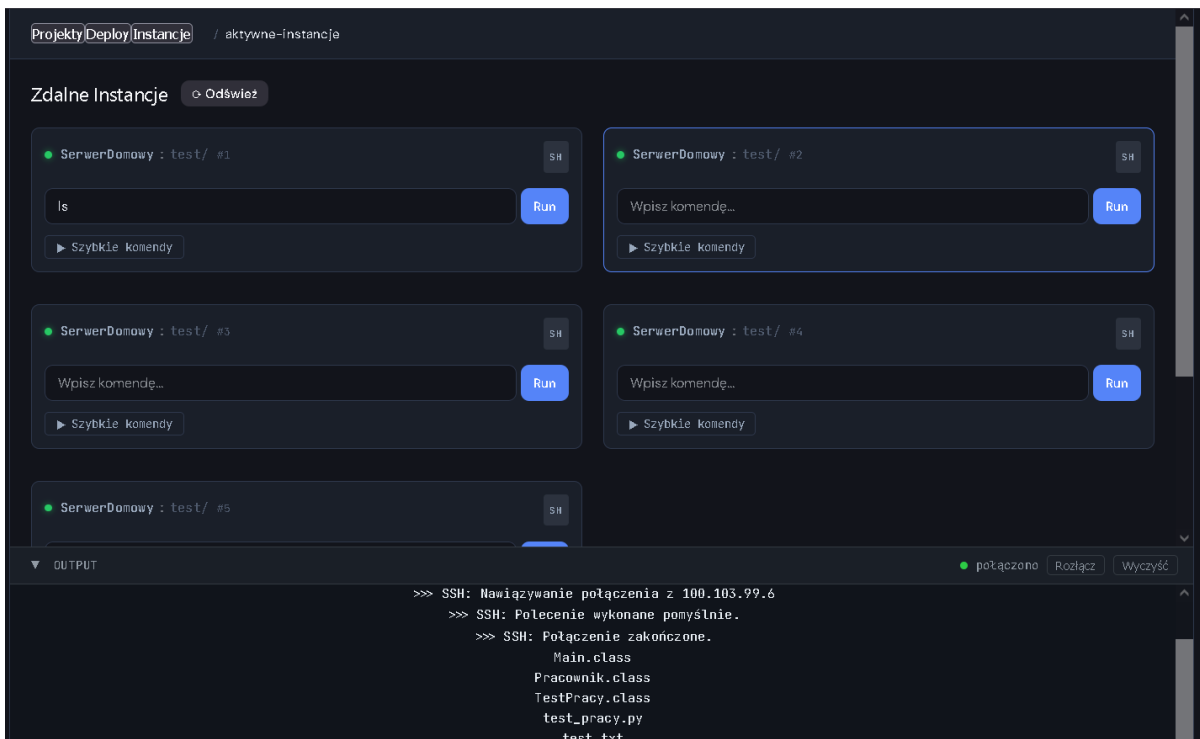
Edytor kodu – widok edycji projektu z listą plików w panelu bocznym, edytorem kodu z podświetlaniem składni oraz przyciskami Zapisz / Kompiluj / Run. Na górze widoczny status kompilacji i uruchomienia, na dole panel OUTPUT z logami przesyłanymi w czasie rzeczywistym przez WebSocket.



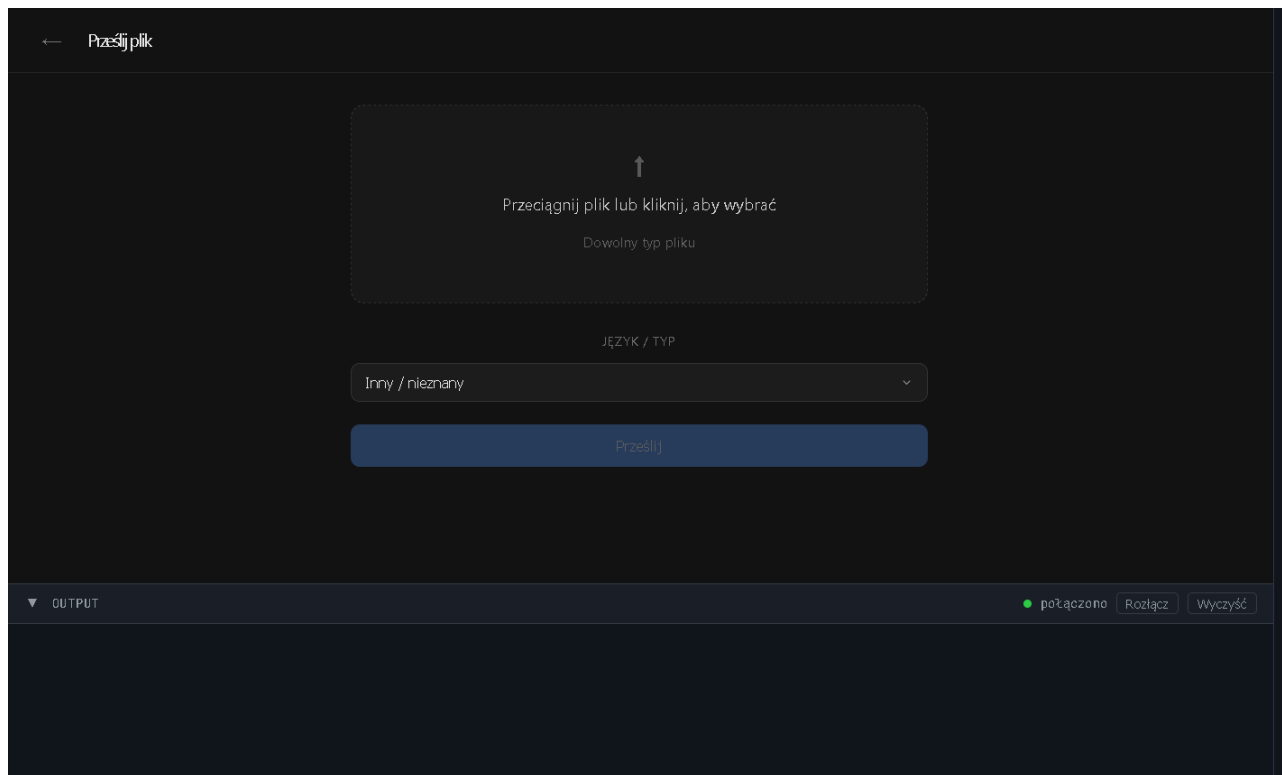
Deploy – widok wdrażania plików na zdalne serwery. Po lewej lista plików źródłowych/skompilowanych/procesów z przypisaną strategią transferu, po prawej lista skonfigurowanych serwerów docelowych, w środku podsumowanie i przycisk Deploy



Instancje – widok zdalnych instancji uruchomionych na serwerach docelowych. Każda karta reprezentuje aktywną instancję z polem do wpisania komendy, przyciskiem Run oraz szybkimi komendami; w panelu OUTPUT widoczne logi z wykonania polecenia przez SSH



Prześlij plik – widok uploadu plików wykonywalnych. Obszar drag&drop do wyboru pliku, wybór języka/typu z listy oraz przycisk Prześlij.



16. Wnioski i możliwości dalszego rozwoju

Projekt BackRunner stanowi kompletny szkielet platformy typu „cloud IDE” z funkcjonalnością zdalnego wdrażania, łączący w sobie kompilację/wykonanie kodu, transfer plików przez SSH oraz komunikację w czasie rzeczywistym przez WebSocket. Zastosowane wzorce projektowe (Strategy, Factory, Facade) zapewniają łatwą rozszerzalność o nowe języki programowania oraz nowe strategie wdrażania.

Możliwe kierunki dalszego rozwoju:

- Zastąpienie bazy H2 trwałą bazą produkcyjną (np. PostgreSQL) oraz dodanie migracji schematu (Flyway/Liquibase).
- Dodanie uwierzytelniania i autoryzacji użytkowników (obecnie kontrolery oznaczone `@CrossOrigin(origins = "*")` są w pełni otwarte).
- Ujednolicenie zduplikowanych kontrolerów testowych (`controller.test.*`) z głównym `CodeController/DeployController`.
- Rozszerzenie `LanguageHandler` o kolejne języki (np. Go, Rust, Node.js) zgodnie z istniejącym wzorcem Strategy.
- Dodanie zarządzania cyklem życia procesów zdalnych (`start/stop/status`) w module `Execution`, obecnie ograniczonego do wysyłania pojedynczych poleceń.